Ulster University

COM 769: Scalable Advanced Software Solutions

Architecting the Interface: Advanced RESTful Web Endpoints

An interactive evaluation of high-level architectural trade-offs, distributed system constraints, and API lifecycle management.

PHASE 1: ARCHITECTURAL FOUNDATIONS

Dilemma 01: The Protocol Fallacy

A legacy enterprise system relies on SOAP over HTTP. During a modernisation meeting, a junior engineer argues that migrating to REST simply means replacing the XML payloads with JSON and continuing to use the HTTP transport layer.

As the Lead Architect, how do you critically evaluate this assertion?

A It is completely accurate; REST is defined entirely by its use of HTTP verbs (GET, POST, PUT, DELETE) and JSON payloads.

B It is partially accurate, but REST mandates the use of HTTPS for security, whereas SOAP does not.

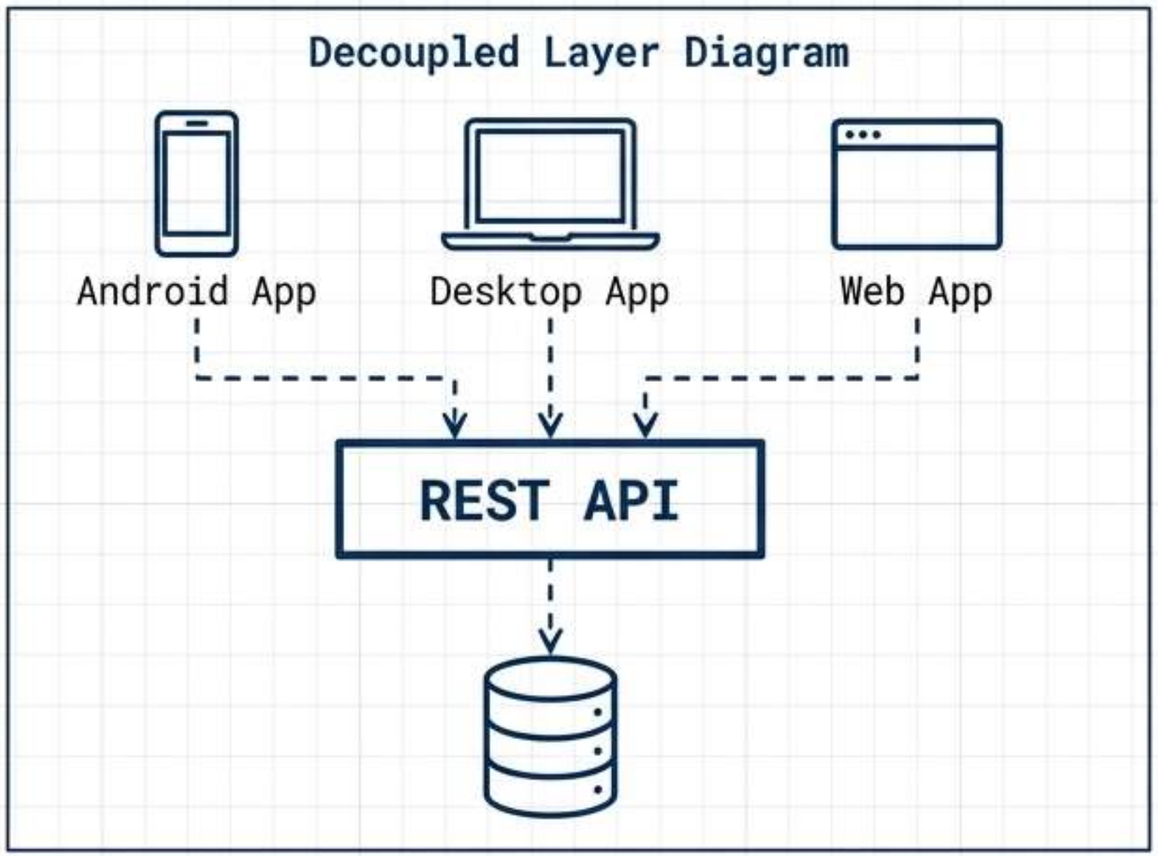
C It is flawed; REST is a protocol-independent architectural style for distributed systems, relying on agreed exchange formats rather than binding to specific transport implementations.

D It is flawed; REST fundamentally prohibits the use of HTTP, requiring custom hypermedia protocols for state transfer.

E It is accurate, provided the system strictly implements the Richardson Maturity Model Level 0.

Decision 01: Protocol Independence

C) It is flawed; REST is a protocol-independent architectural style for distributed systems, relying on agreed exchange formats rather than binding to specific transport implementations.



Proposed by Roy Fielding in 2000,

REST (Representational State Transfer) is an architectural style, not a protocol. While most common implementations use HTTP, REST is protocol-agnostic.

Key Constraint:

The primary advantage is platform independence. By leveraging standard exchange formats (like JSON) and open standards, neither the client nor the server is bound to a specific internal implementation.

PHASE 1: ARCHITECTURAL FOUNDATIONS

Dilemma 02: Horizontal Scale

An e-commerce API is experiencing severe latency during peak traffic. The current architecture uses a load balancer with "sticky sessions" (server affinity) to ensure a user's multi-step checkout requests hit the same server holding their session data.

To scale out horizontally, horizontally, which RESTful constraint must be enforced, and what is its direct impact?

A Enforce Uniform Interface; the load balancer must rewrite all requests to use the PATCH verb to update session state efficiently.

B Enforce HATEOAS; the server must send the entire database schema to the client on the first request to eliminate future lookups.

C Enforce the Stateless constraint; session data must be retained in memory on all servers simultaneously via continuous replication.

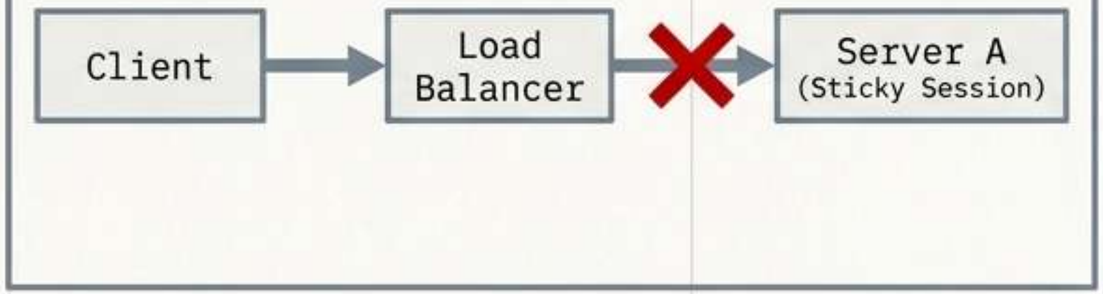
D Enforce the Stateless constraint; requests must be independent and atomic, storing all necessary state within the resources themselves, entirely removing the need for server affinity.

E Enforce Client-Server separation; the client must take over the load balancing duties, directing traffic based on MAC addresses.

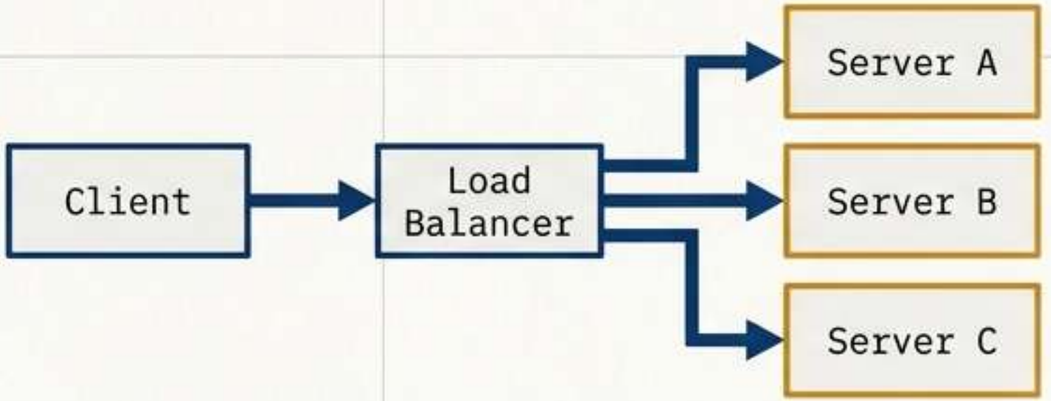
Decision 02: The Stateless Constraint

D) Enforce the Stateless constraint; requests must be independent and atomic, storing all necessary state within the resources themselves, entirely removing the need for server affinity.

Anti-Pattern



RESTful Pattern



REST APIs operate on a stateless request model. Keeping transient state between requests makes scaling unfeasible. The only place information is stored is in the resources themselves. Each request must be independent and atomic. Because no server affinity is required, any server can seamlessly handle any request from any client.

Dilemma 03: Database Entanglement

A backend development team is designing a new REST API for an inventory system. To save time, they create endpoints that exactly match their internal relational database tables: `/dbo-inventory-items`, `/dbo-supplier-links`, and `/dbo-stock-levels`.

Why is this approach fundamentally flawed in RESTful design?

A It fails to utilise plural nouns for the collections, breaking the uniform interface constraint.

B It tightly couples the client to the internal implementation, breaking the abstraction layer and making database evolution impossible without breaking client applications.

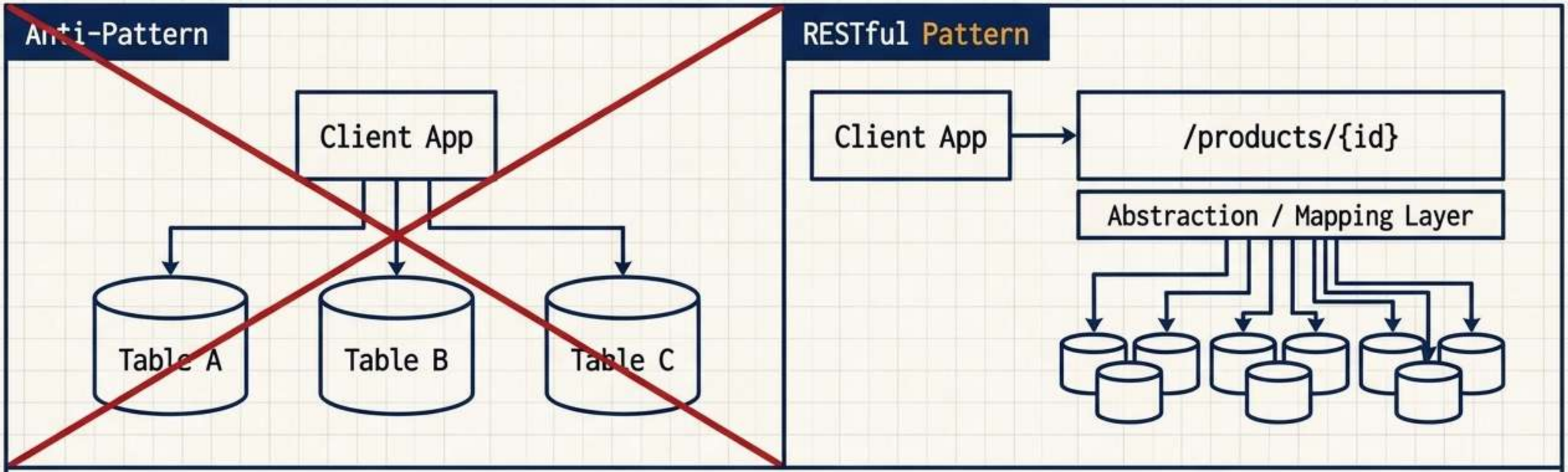
C It increases the latency of the API because SQL table names take longer to parse in URI routing frameworks.

D It is an acceptable practice if the API uses the PUT method to create new relational schemas on the fly.

E It violates the HATEOAS constraint by not including a `.sql` extension at the end of the URI.

Decision 03: Resource Abstraction

B) It tightly couples the client to the internal implementation, breaking the abstraction layer and making database evolution impossible without breaking client applications.



A web API is an abstraction of the database, not a mirror of it. An entity might span multiple relational tables internally, but must be presented to the client as a single logical resource. Introducing a mapping layer insulates client applications from backend structural changes, preserving service evolution.

PHASE 2: STRUCTURAL DESIGN & RESOURCE MODELLING

Dilemma 04: The Infinite Path

You are reviewing a pull request that introduces the following URI structure to retrieve a specific item from a specific order belonging to a specific customer:

/customers/5/orders/99/items/4.

According to RESTful best practices for managing complex relationships, what is the most robust architectural critique of this URI?

A It is too complex and cumbersome; relationships extending beyond collection/item/collection should be managed via navigable links in the response body rather than deep URI nesting.

B It uses plural nouns incorrectly; the correct path should be /customer/5/order/99/item/4.

C It is perfectly designed, representing Richardson Maturity Model Level 3.

D It requires the client to issue a POST request to navigate the path, violating idempotency.

E It exposes the primary keys of the database, which is explicitly forbidden by the statelessness constraint.

Decision 04: Handling Relationships

A) It is too complex and cumbersome; relationships extending beyond collection/item/collection should be managed via navigable links in the response body rather than deep URI nesting.

The Architectural Rule

Tip: Avoid requiring resource URIs more complex than collection/item/collection.

`/customers/5/orders`

The Solution

```
{
  "orderID": 3,
  "productID": 2,
  "quantity": 4,
  "orderValue": 16.60,
  "links": [
    { "rel": "product",
      "href":"https://adventure-works.com/customers/3",
      "action": "GET" },

    { "rel": "product",
      "href":"https://adventure-works.com/customers/3",
      "action": "PUT" }
    ]
}
```

Extending the path model too far becomes cumbersome to implement and fragile to maintain. The superior solution is providing **navigable links to associated resources directly** within the HTTP response message.

PHASE 3: OPERATIONAL MECHANICS & STATE MANAGEMENT

Dilemma 05:
The Granularity Balance

A mobile application requires 15 distinct GET requests to different fine-grained endpoints just to render its home dashboard. The engineering team proposes denormalising the data into a single, massive “dashboard” resource.

What is the primary architectural trade-off of this consolidation?

A It reduces server load and latency, but entirely prevents the use of standard HTTP verbs.

B It decreases the number of requests, but increases the risk of over-fetching data the client doesn’t need, potentially incurring higher bandwidth costs.

C It improves caching efficiency at the edge, but requires the API to abandon JSON in favour of XML.

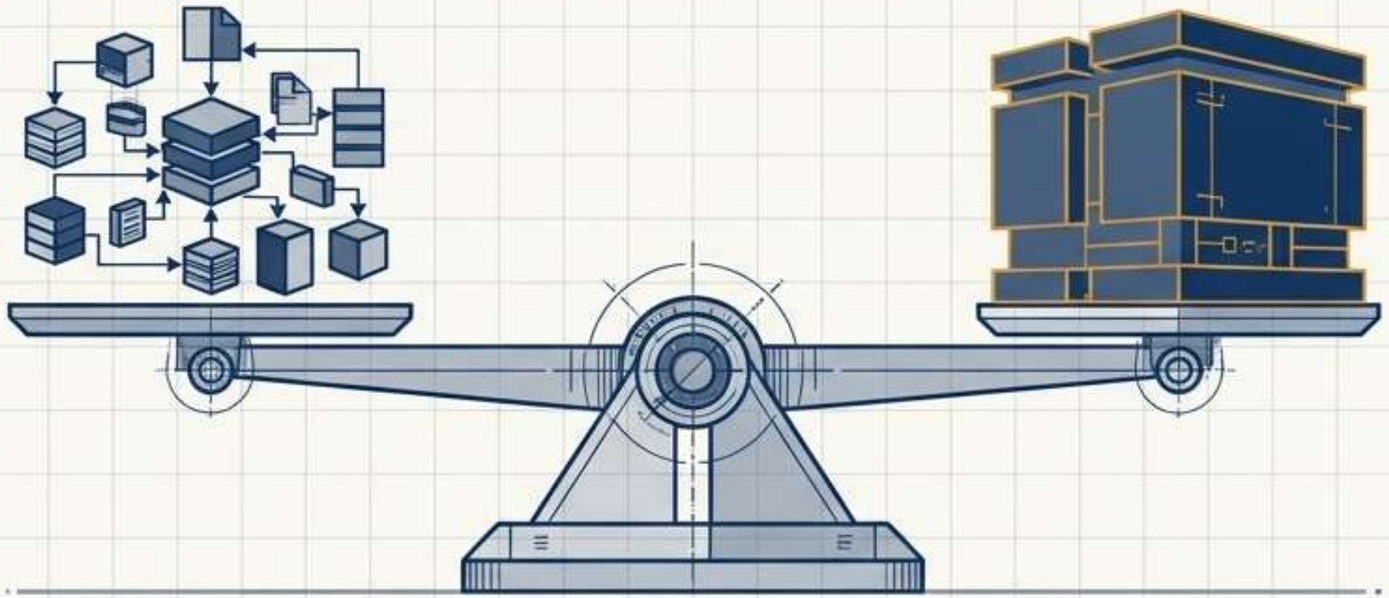
D It violates the platform independence constraint by tailoring the payload specifically for a mobile device.

E It forces the server to retain state between requests, breaking the stateless constraint.

Decision 05: API Granularity

B) It decreases the number of requests, but increases the risk of over-fetching data the client doesn't need, potentially incurring higher bandwidth costs.

Chatty APIs (High Request Volume)



Chunky APIs (Large Resource Payloads)

Trade-off: Increased Server Load & Total Round-Trip Latency

Trade-off: Over-fetching & Increased Bandwidth Costs

Web requests impose load. A 'chatty' API exposing many small resources causes unacceptable latency. Denormalising data into larger resources solves this, but must be carefully balanced against bandwidth costs and fetching irrelevant data.

PHASE 3: OPERATIONAL MECHANICS & STATE MANAGEMENT

Dilemma 06: The Temporal Constraint

A REST operation triggers a massive background data aggregation that takes roughly 4 minutes to complete. Holding the HTTP connection open causes unacceptable latency and client timeouts.
How should the API handle this request asynchronously while remaining RESTful?

A) Return a 504 Gateway Timeout immediately, forcing the client to continuously retry the original POST request.

B) Open a persistent WebSockets connection on the same URI to stream the processing logs in real-time.

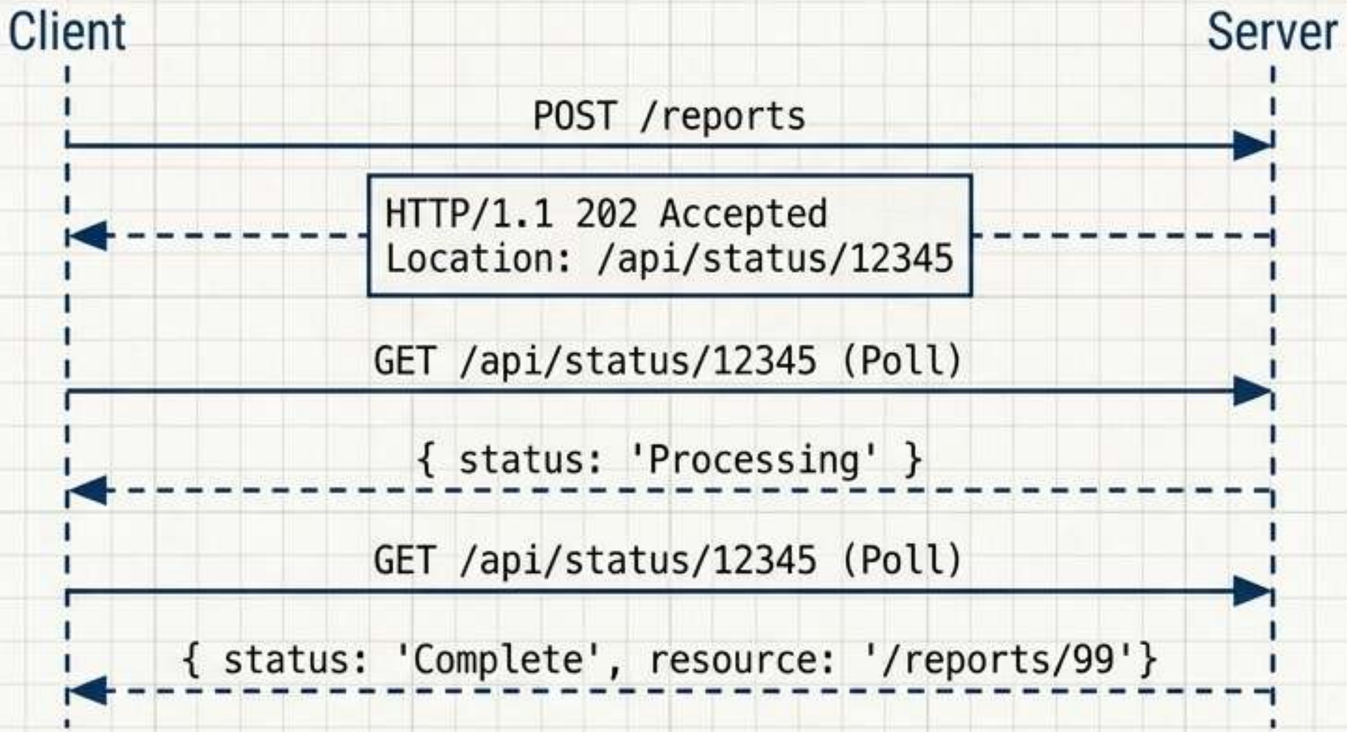
C) Return a 202 Accepted status code with a `Location` header containing the URI of a status-monitoring endpoint.

D) Return a 200 OK status code immediately with an empty JSON body, assuming the process will eventually succeed.

E) Redirect the client using a 301 Moved Permanently status to a new server dedicated entirely to slow requests.

Decision 06: Asynchronous Processing

C) Return a 202 Accepted status code with a Location header containing the URI of a status-monitoring endpoint.



Avoid holding connections open for long operations. Accepting the request (202) and providing a dedicated status URI allows the client to monitor progress via polling without blocking system resources.

PHASE 4: HYPERMEDIA & LONG-TERM EVOLUTION

Dilemma 07: Lifecycle Mutability

The engineering team must deploy a breaking schema change to the customer resource. You need a versioning strategy that explicitly routes requests to the new schema while allowing older client applications to hit the exact same endpoints they always have, without requiring them to inject new HTTP headers. Which strategy achieves this?

A) Query String Versioning (e.g., `?version=2`).

B) Content-Type Negotiation (Accept header versioning).

C) URI Versioning (e.g., `/v2/customers/3`).

D) Resource renaming (e.g., `/customers-new/3`).

E) Dropping backward compatibility entirely, as REST mandates clients adapt dynamically.

Decision 07: Versioning Strategies

C) URI Versioning (e.g., /v2/customers/3).

URI Versioning	Query String	Header Versioning
`https://adventure-works.com/v2/customers/3`	`https://adventure-works.com/customers/3?version=2`	`Custom-Header: api-version=1`
• Explicit routing, clear visually, but changes fundamental resource ID.	• Easy to implement, defaults cleanly to v1 for legacy clients.	• Keeps URIs perfectly clean, but requires client-side HTTP manipulation.

APIs are never static. Service evolution dictates that as schemas change, existing clients must function without modification. Versioning is the ultimate safeguard of a scalable architecture.

PHASE 1: ARCHITECTURAL FOUNDATIONS

Dilemma 08: The Adapter Fallacy

A microservice ecosystem has 50 different endpoints, each requiring custom JSON payloads to dictate actions (e.g., `{'action': 'create_user'}`, `{'action': `{'action': 'modify_status'}`) all sent via HTTP POST.
How does enforcing the RESTful Uniform Interface constraint resolve this tight coupling?

A) It mandates XML instead of JSON, which automatically structures all internal payloads.

B) It relies on standard HTTP verbs (GET, POST, PUT, PATCH, DELETE) to define operations, acting as a generic, universally understood interface between systems.

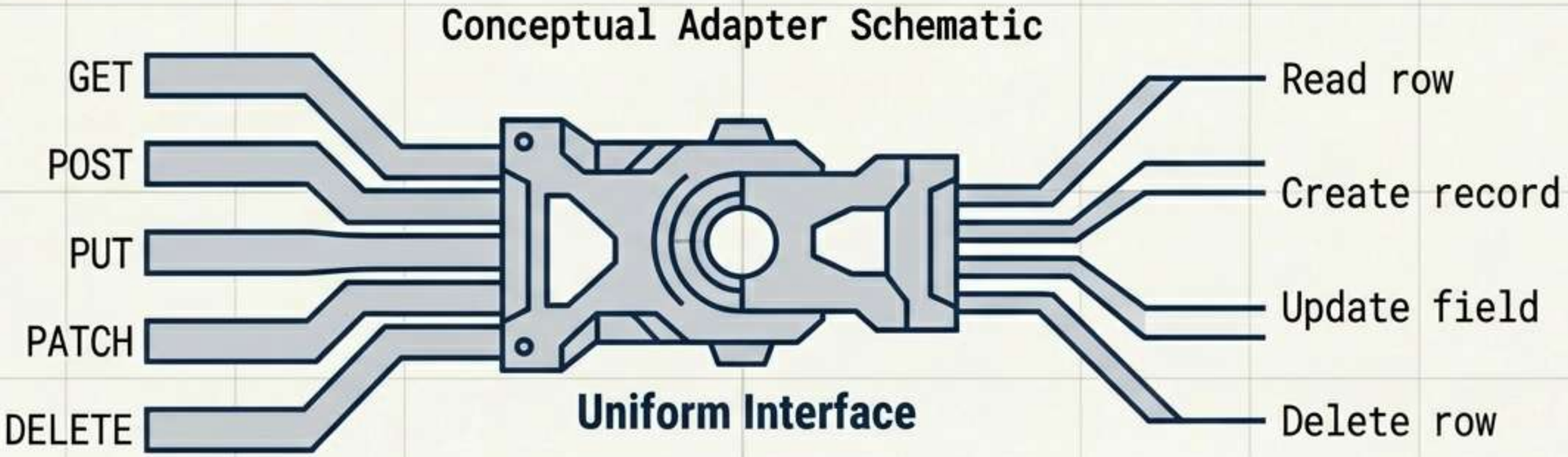
C) It forces all requests to pass through a single, master `/api` route, eliminating custom endpoints entirely.

D) It requires the client to download the server's source code before issuing requests.

E) It replaces HTTP verbs with complex query parameters to mask the operations.

Decision 08: The Uniform Interface

B) It relies on standard HTTP verbs (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`) to define operations, acting as a generic, universally understood interface between systems.



REST APIs use a uniform interface, which heavily decouples the client and service implementations. Instead of inventing proprietary action schemas inside JSON payloads, the API leverages the standard application protocol operations. Any compliant client instantly understands the intent of the transaction.

PHASE 1: ARCHITECTURAL FOUNDATIONS

Dilemma 09: Technological Isolation

A desktop client written in C# and an iOS mobile application written in Swift both need to consume the same web API, which is built entirely in Node.js on a Linux server.

What fundamental REST principle guarantees they can seamlessly interact without having to understand the server's backend languages or internal memory structures?

A) Platform Independence via the exchange of standard representations (like JSON).

B) The HATEOAS constraint, which translates backend code into Swift automatically.

C) Resource Abstraction, which forces the database to run locally on the client.

D) Versioning, which creates separate identical APIs for every operating system.

E) Asynchronous Processing, preventing the clients from blocking the server.

Decision 09: Platform Independence

A) Platform Independence via the exchange of standard representations (like JSON).

A well-designed web API supports platform independence. Clients interact with a service by exchanging representations of resources, using open standards and mechanisms to agree on the format of the data (such as JSON). The client is oblivious to how the API is implemented internally.

Dilemma 10: Naming Conventions

A developer proposes the following endpoints for managing customer profiles:

`POST /create-customer`, `GET /get-customer/5`, and `POST /update-customer/5`.

Based on core REST conventions, why is this an architectural anti-pattern?

A) Resource URIs should be based on nouns (the entities), not verbs. The HTTP method itself defines the operation.

B) The endpoints are too chatty and must be combined into a single /manage-customer` URI.

C) Pluralization is incorrect; it should be /create-customers`.

D) It lacks hypermedia state transfer links embedded inside the paths.

E) It is entirely correct; embedding verbs in the path makes routing faster.

Decision 10: Nouns vs Verbs

A) **Resource URIs should be based on nouns** (the entities), not verbs. The HTTP method itself defines the operation.

✗	Avoid: Verb-based Routing	✓	Good: Collection/Item Hierarchy
✗	POST /create-customer	✓	POST /customers
✗	GET /get-customer/5	✓	GET /customers/5
✗	POST /update-customer/5	✓	PUT /customers/5

Organise the API design around business entities (collections and items). The resource identifier acts purely as a locator (a noun). The action performed on that resource is determined strictly by the HTTP verb applied to it.